CAPÍTULO 3. RAMIFICACIONES EN GIT

Introducción

Las ramificaciones, también conocidas como *branches*, son una de las características más importantes de Git, ya que permiten trabajar en diferentes versiones de un proyecto sin modificar directamente la versión principal. Gracias a ellas, es posible desarrollar nuevas funcionalidades, corregir errores y experimentar con cambios de manera organizada.

En Git, una rama es una línea independiente de desarrollo. Esto significa que se pueden realizar modificaciones en una rama sin afectar las demás. Cuando los cambios están listos y han sido revisados, pueden integrarse en otra rama mediante un proceso denominado fusión o *merge*.

El objetivo de este capítulo es aprender a crear, consultar, cambiar, fusionar y eliminar ramas, así como comprender los conflictos que pueden presentarse al integrar cambios.

1. ¿Qué es una rama en Git?

Una rama representa una línea de desarrollo independiente dentro de un repositorio. En lugar de crear una copia completa del proyecto para trabajar en una nueva funcionalidad, Git permite crear una rama que parte de un punto específico del historial de cambios.

Por ejemplo, un proyecto puede tener una rama principal llamada main y otras ramas destinadas a tareas específicas, como corregir errores o agregar funcionalidades.

Al trabajar con ramas, se pueden mantener organizados los cambios y evitar que las modificaciones que todavía están en desarrollo afecten la versión principal del proyecto.

2. Crear una rama

Para crear una rama nueva se utiliza el comando git branch, seguido del nombre que se desea asignar.

Comando:

```
git branch nombre-rama
```

Ejemplo:

```
git branch nueva-funcionalidad
```

Este comando crea una rama llamada nueva-funcionalidad, pero no cambia automáticamente a ella. Para consultar las ramas existentes y verificar cuál está activa, se utiliza:

```
git branch
```

Git muestra las ramas disponibles en el repositorio. La rama actual aparece identificada con un asterisco (*).

3. Cambiar de rama

Para comenzar a trabajar en una rama diferente, se puede utilizar el comando git switch.

Ejemplo:

```
git switch nueva-funcionalidad
```

Este comando cambia la rama activa a nueva-funcionalidad. A partir de ese momento, los nuevos cambios y confirmaciones se realizarán en esa rama.

También es posible crear una rama y cambiar a ella en un solo paso:

```
git switch -c nueva-rama
```

El comando crea la rama nueva-rama y la convierte inmediatamente en la rama activa.

En versiones anteriores de Git, era habitual utilizar git checkout para cambiar de rama o crear una nueva. Sin embargo, git switch ofrece una forma más clara de realizar esta operación.

4. Realizar cambios en una rama

Una vez creada y seleccionada una rama, se pueden modificar archivos, agregar nuevos documentos y registrar los cambios mediante confirmaciones o *commits*.

Por ejemplo, para crear un archivo llamado funcionalidad.txt, se puede utilizar el siguiente comando en una terminal compatible con estos comandos:

```
echo "Nueva funcionalidad del proyecto" > funcionalidad.txt
```

Después, se agrega el archivo al área de preparación:

```
git add funcionalidad.txt
```

Finalmente, se registra el cambio:

```
git commit -m "Agrega nueva funcionalidad"
```

La confirmación queda registrada en la rama activa. Por esta razón, es importante verificar en qué rama se está trabajando antes de realizar modificaciones.

Para consultar la rama actual, se puede ejecutar:

```
git branch
```

5. Fusionar ramas

La fusión, conocida como *merge*, permite integrar los cambios de una rama en otra. Es especialmente útil cuando una funcionalidad ya está terminada y se desea incorporarla a la rama principal del proyecto.

Para realizar una fusión, primero se debe cambiar a la rama que recibirá los cambios. Por ejemplo, si se quiere integrar nueva-funcionalidad en main, se ejecutan los siguientes comandos:

```
git switch main

git merge nueva-funcionalidad
```

El primer comando cambia a la rama main. El segundo integra en ella los cambios de nueva-funcionalidad.

Si Git puede combinar los cambios automáticamente, la fusión se completa sin inconvenientes. Dependiendo del historial del proyecto, Git puede realizar una fusión rápida (*fast-forward*) o crear una confirmación de fusión (*merge commit*).

Es recomendable revisar el estado del repositorio después de fusionar ramas y comprobar que los archivos contengan los cambios esperados.

6. Resolver conflictos de fusión

Un conflicto ocurre cuando Git no puede integrar automáticamente determinados cambios. Esto puede suceder, por ejemplo, cuando dos ramas modifican la misma parte de un archivo de formas incompatibles.

Cuando aparece un conflicto, Git señala las partes que necesitan revisión. En el archivo afectado pueden aparecer marcas como las siguientes:

```
<<<<<<< HEAD

Texto de la rama actual

=======

Texto de la otra rama

>>>>>>> nueva-funcionalidad
```

Estas marcas permiten identificar las versiones en conflicto. Para resolverlo, se deben seguir estos pasos:

1. Abrir el archivo afectado y revisar las dos versiones del contenido.
2. Decidir qué contenido debe conservarse, o combinar las dos versiones según las necesidades del proyecto.

3. Eliminar las marcas de conflicto y dejar el contenido final correctamente organizado.
4. Guardar el archivo y agregarlo al área de preparación.

```
git add nombre-archivo.txt
```

5. Completar la fusión mediante una confirmación, si Git la solicita:

```
git commit
```

La resolución de conflictos requiere revisar cuidadosamente el contenido para evitar perder información importante.

7. Eliminar ramas

Después de fusionar una rama y comprobar que sus cambios fueron incorporados correctamente, es posible eliminarla si ya no se necesita.

Para eliminar una rama local que ya fue fusionada, se utiliza:

```
git branch -d nueva-funcionalidad
```

La opción -d permite eliminar una rama fusionada de forma segura. Si la rama contiene cambios que no se han integrado, Git normalmente impedirá eliminarla con esta opción.

Existe también la opción -D, que fuerza la eliminación:

```
git branch -D nueva-funcionalidad
```

Este comando debe utilizarse con precaución, ya que puede eliminar una rama que contenga trabajo todavía no integrado.

Para consultar las ramas existentes antes de eliminarlas, se recomienda ejecutar:

```
git branch
```

8. Consultar el historial de ramas

Git permite revisar el historial de confirmaciones para comprender cómo ha evolucionado el proyecto y qué cambios se han realizado.

Para consultar el historial de manera resumida, se utiliza:

```
git log --oneline
```

Si se desea visualizar las ramas y las relaciones entre las confirmaciones, se puede ejecutar:

```
git log --oneline --graph --all
```

Este comando muestra una representación gráfica del historial, incluyendo las diferentes ramas y las fusiones realizadas. Es útil para comprender cómo se han integrado los cambios a lo largo del desarrollo.

9. Ejercicio práctico

Para poner en práctica los conceptos de este capítulo, se realizará un ejercicio en un repositorio Git existente.

Paso 1. Consultar las ramas disponibles:

git branch

Paso 2. Crear una rama para una nueva funcionalidad:

git switch -c ejercicio-ramas

Paso 3. Crear un archivo de prueba:

echo "Práctica de ramificaciones en Git" > practica.txt

Paso 4. Agregar el archivo al área de preparación:

git add practica.txt

Paso 5. Registrar el cambio:

git commit -m "Agrega archivo de práctica"

Paso 6. Regresar a la rama principal:

git switch main

Paso 7. Fusionar la rama del ejercicio:

git merge ejercicio-ramas

Paso 8. Revisar el historial de cambios:

git log --oneline --graph --all

Al finalizar, el archivo practica.txt deberá estar integrado en la rama main. Si el repositorio utiliza otro nombre para su rama principal, como master, se debe reemplazar main por el nombre correspondiente.

Conclusión

Las ramificaciones permiten desarrollar proyectos de manera organizada, ya que hacen posible trabajar en funcionalidades diferentes sin modificar directamente la línea principal de desarrollo. Los comandos para crear, cambiar, fusionar y eliminar ramas son fundamentales para administrar adecuadamente un repositorio Git.

Asimismo, comprender el proceso de fusión y la resolución de conflictos facilita la integración de cambios realizados por diferentes personas o en distintas etapas de un proyecto. El uso correcto de las ramas mejora la organización, el control de versiones y la colaboración durante el desarrollo de software.

Referencias

Chacon, S., y Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. Capítulo 3: Ramificaciones en Git. Disponible en: https://git-scm.com/book/es/v2/Ramificaciones-en-Git-Conceptos-de-Ramificaciones